VPTunnel A Kernel Space Site to Site VPN

Aayush Kuloor *24110004* | Anvit Aggarwal *24110048* | Dhru Harsh Darshakbhai *24110107* | Raj Amit Modi *24110282*

CS331 Computer Networks Team T005 Project 1 IIT Gandhinagar

I. Introduction

Private services at separate sites should be reachable only through an authenticated, explicitly authorized path. Encryption alone does not supply that boundary: a gateway must also distinguish a permitted initiation from an unsolicited reply, reject replayed records, handle IP forwarding correctly, and stop admitting traffic when its control plane fails.

VPTunnel implements a routed, Layer-3, site-to-site IPv4 VPN. A Rust controller authenticates pinned peers using Noise IK and hands directional keys to a Linux kernel module. The kernel owns every application-data record: encryption, authentication, counters, replay state, policy and packet delivery. The controller never relays application packets after key handoff. Two ordinary LAN hosts can communicate by private IP address through the gateways, subject to a default-deny access policy.

The project contributes an explicit control/data-plane split, an authenticated record format with a 72-byte IPv4 encapsulation cost, transactional policy updates, bounded fragment processing, flow-state helper modules, and reproducible namespace and Raspberry Pi experiments. Physical Pi-to-Pi TCP measurements deliver approximately 895 Mbit/s over a gigabit cable, retaining 95.5% of direct goodput. Matched native namespace tests retain approximately 95% through a 1 Gbit/s capacity setting; at 2.5 Gbit/s the tunnel becomes limited below the configured capacity. These are distinct experimental environments.

II. Problem Statement

Given two non-overlapping private LANs, a UDP-reachable underlay, pinned gateway public keys, routes and an initiation policy, forward authorized TCP, UDP and ICMP echo traffic between the LANs. Output consists of authenticated encrypted UDP records on the underlay and validated inner IPv4 packets at the destination site. A denied or invalid packet must produce no application delivery. Loss of the controller's five-second lease must close admission.

The model assumes trusted gateways, protected private keys, correct local routing and no inner or outer NAT. An attacker may observe, inject, modify, reorder, replay or suppress underlay datagrams. Confidentiality does not hide outer addresses, sizes or timing, and the design cannot prevent denial of service. The scope is at most four peers with disjoint prefixes; it excludes inner IPv6, roaming, overlapping-subnet translation, bridging and broadcast discovery. The current packet path applies directional ACLs and reply-tuple checks; full stateful proof of an earlier permitted initiation remains incomplete. Connectivity and cryptographic performance do not establish that stronger policy property.

III. Project Objectives

1. Carry permitted private-IP traffic and keep the sites unreachable with the VPN disabled.
2. Authenticate pinned peers and protect each data record against modification and replay, without nonce reuse.
3. Enforce initiation direction, current policy, generation eligibility and lease expiry at the admission boundary.
4. Bound memory, queues and outstanding work; support rekey, teardown, fragments and MTU-related failures.
5. Measure receiver goodput, idle/loaded RTT, UDP delivery and whole-system/per-core CPU on native and physical testbeds.

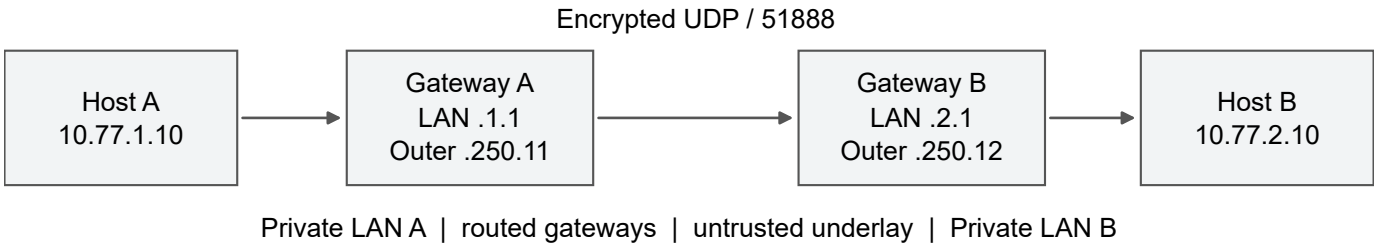


Figure 1. Intended site-to-site deployment. Abbreviated gateway addresses expand within 10.77.1.0/24, 10.77.2.0/24 and 172.31.250.0/24. The private LANs remain separate routed subnets.

V. Architecture and Design

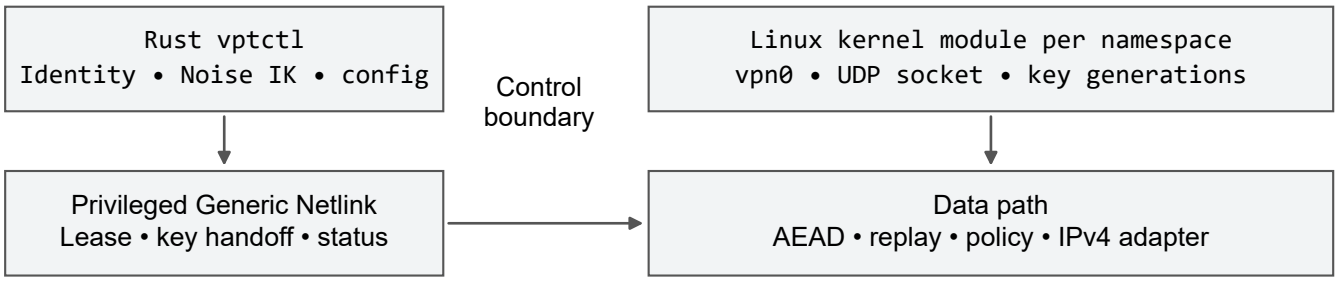


Figure 2. Control and data ownership. Generic Netlink is a privileged, namespace-scoped control boundary. Application packets remain in the kernel.

Component responsibilities

The userspace controller, `vptctl`, validates configuration, protects the static X25519 identity, runs the Noise exchange, renews ownership and requests rekey. It uses `snow` for Noise and makes one Split-key handoff for an establishment attempt. The kernel module, `vpt1_foundation.ko`, creates `vpn0`, binds a UDP socket and owns peer incarnations, generations, sequence counters, ACL snapshots, liveness and runtime accounting. The gateway scripts configure addresses, routes and a small nftables forwarding ruleset; the controller does not take over the system default route.

Netlink transactions bind ownership, configuration revision, peer incarnation, attempt, hash and continuity to the key handoff. Status returns no raw keys. Namespaces have separate gateway state and UDP bindings.

End-to-end packet processing

A LAN host routes a remote private prefix through its gateway. The Linux route selects `vpn0`. The transmit path validates IPv4, normalizes fragments when necessary, selects the owning peer, checks policy and generation eligibility, and reserves a counter before sealing the record. A final transmit recheck prevents output under state invalidated while encryption was running. Sending failure never refunds the counter.

The receiver checks framing and the configured outer source, resolves the receive locator to a retained generation, and performs a read-only replay precheck. AEAD authentication precedes the locked replay claim. Authenticated control records enter the lifecycle state machine; application data requires a committed generation. Inner IPv4 and transport checks precede final admission. The recovered packet is scrubbed of outer metadata and injected on `vpn0` for normal stack routing to the LAN.

Design choices and tradeoffs

UDP preserves record boundaries and avoids an outer reliable byte stream; inner TCP remains responsible for retransmission. An L3 device carries IP traffic without extending Ethernet broadcasts between sites. A userspace handshake keeps identity and exchange logic out of the fast path, while synchronous kernel AEAD avoids a userspace crossing per packet. This split reduces data-path context switching but introduces kernel lifetime, locking and allocation obligations.

Pinned identities simplify authentication but require manual provisioning and static endpoints. Immutable RCU snapshots, peer locks and generation references make policy publication and packet lifetimes explicit. Bounded peers, policy and memory favor predictable behavior. The unrestricted direct path benefits from virtual aggregation/offloads that do not remove VPN per-record work.

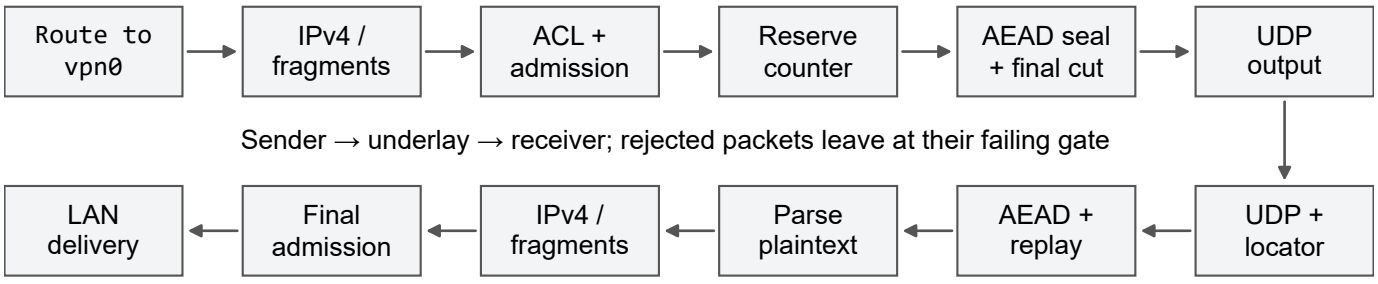


Figure 3. Application packet pipeline. The receiver claims a replay counter only after authentication, then validates plaintext and authorization. Fragment work is bounded and precedes transport-policy inspection.

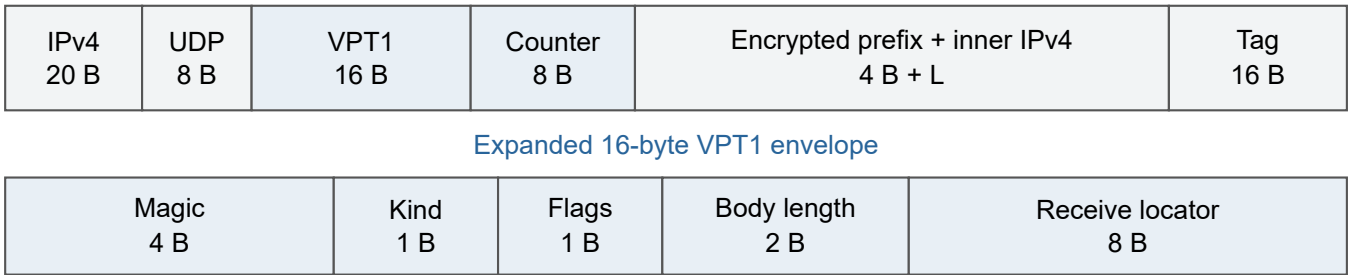


Figure 4. Protected record format; widths are schematic. The 16-byte envelope contains magic[4], kind[1], flags[1], body length[2] and receive locator[8]. Envelope plus counter form the 24-byte AAD.

Parser bytes 6–7 carry body length. Exact framing, zero flags and supported kinds/subtypes are required. A collision-checked random locator selects a receive generation, not an identity. ChaCha20-Poly1305 uses a 32-byte key, 16-byte tag and nonce $0^{32} \| \mathrm{LE64}(c)$ [2]. Data/control share counters, reserved before encryption without refunds.

Encapsulation adds 72 bytes: outer IPv4 20, UDP 8, envelope 16, counter 8, encrypted prefix 4 and tag 16. A 1500-byte outer MTU therefore permits 1428-byte inner IPv4 or 1400-byte inner UDP payload. Outer fragmentation is prohibited; inner normalization/re-fragmentation is separate.

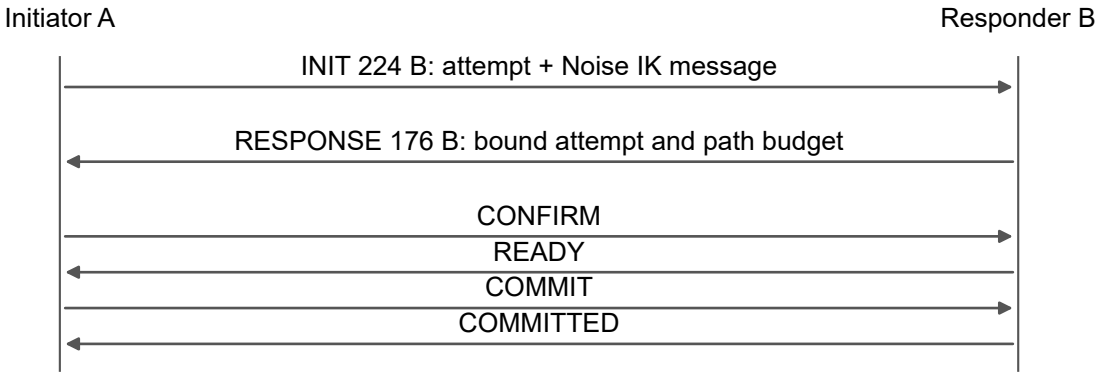


Figure 5. Pinned Noise IK establishment and protected kernel activation. INIT is envelope[16] + attempt[16] + Noise[192]; RESPONSE is envelope[16] + attempt[16] + Noise[144]. Activation denotes four sequential protected messages.

An 80-byte prologue binds roles and static keys, with BLAKE2s hashing; the exchange authenticates attempts, locators, incarnation and path budget. All-zero X25519 results and incompatible budgets fail establishment. Noise Split gives initiator TX/RX=k_1/k_2, reversed at the responder [1]. Fresh IK supplies every new key; userspace never allocates transport nonces.

VI. Implementation

Kernel C, Rust 2024, Python tools and shell scripts implement the design. Rust 1.93.1 pins `snow` 0.10.0 and forbids unsafe code. Kernel AEAD is synchronous `rfc7539(chacha20,poly1305)`. Source: VPTunnel repository.

Receive processing follows four gates: (1) parse framing, locator and static source; hold references; (2) replay-precheck, authenticate, then atomically claim counter/budget; (3) dispatch control or validate committed-generation IPv4 and current admission; (4) scrub metadata, deliver and release references. The 1024-bit replay window is per generation/direction. Failed authentication cannot advance it; authenticated malformed plaintext consumes its claimed counter.

Current ACLs match prefixes, protocol, ports and original/reversed tuples. TCP reverse traffic requires ACK/RST. This stateless gate does not prove earlier initiation: separate TCP/UDP/echo flow-state helpers are not called by the measured fast path. Bounded fragment workers normalize before inspection; the IP adapter validates headers/checksums, ECN, MTU/DF and correlated ICMP errors. Reserved gateway .1 addresses require separate .10 test aliases.

Candidates carry control until four-message commit; old RX keys have ten-second grace. Age or 300 MiB/300,000 device records trigger rekey; hard limits are 2^{20} records or 1 GiB per direction. Admission closes on five-second lease or fifteen-second liveness expiry. Bounds include four peers, 128 rules, 64 queued jobs and 4096 in-flight admissions/peer; accounted memory caps are 16 MiB/gateway and 128 MiB/host. Teardown drains work/references and erases keys. Additional diagrams accompany the sources.

VII. Testbed Setup

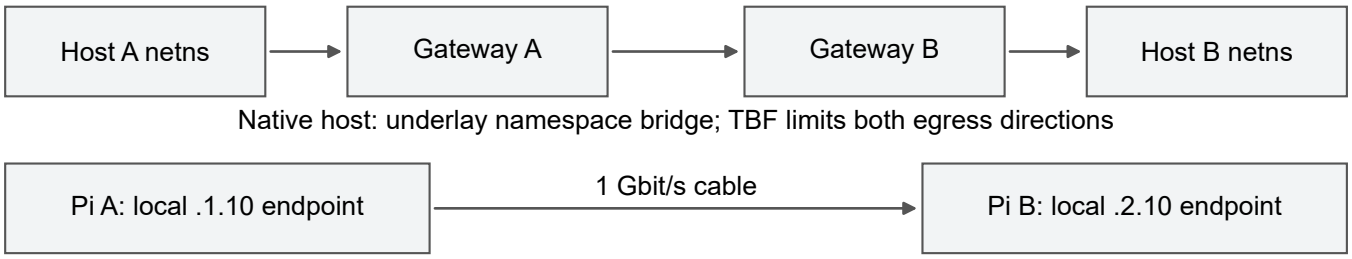


Figure 6. The measured native namespace path and physical Pi path. Native traffic traverses separate host and gateway namespaces on one CPU system. Physical traffic originates on each Pi; Wi-Fi is used only for SSH management.

Native namespace environment

The native machine is an Intel Core Ultra 7 165H with 22 logical CPUs and approximately 30 GiB usable RAM, running x86-64 Linux `6.18.34-061834-generic` without virtualization. It uses AC power and the powersave governor, without CPU pinning or frequency locking. Metadata records iperf3 3.16, GCC 14.2 and iproute2.

Five namespaces represent host A, gateway A, an underlay bridge, gateway B and host B. Baseline forwards between the same hosts without encapsulation; tunnel routes through `vpn0`. Unrestricted tests have no physical link limit. Original and CPU-instrumented sessions remain separate.

For matched capacities, TBF shapes both underlay bridge egress ports at 100, 500, 1000 or 2500 Mbit/s. Burst is one millisecond of capacity with a 16 KB minimum, queue latency is 20 ms, accounting overhead is 24 bytes and minimum packet unit is 64 bytes. Encapsulated outer traffic consumes that budget. TSO/GSO/GRO are disabled on all experiment veths for both paths; unrestricted tests retain defaults. Thus comparisons across these two lanes also change offload behavior. Mode order alternates by capacity; the documented checkpoint/resume at 100 Mbit/s remains part of the evidence.

Physical gateways

Two Raspberry Pi 5 Model B gateways use four Cortex-A76 cores, up to 2.4 GHz, and native aarch64 Linux `6.18.34+rpt-rpi-2712`. The vendor kernel uses 16 KiB pages. Matching GCC 14 headers and Rust 1.93.1 build the module/controller natively. The formal metadata does not establish board RAM capacity; the benchmark does not require a GPU. Both Ethernet ports negotiate 1000 Mbit/s, full duplex, over a direct cable. The default ondemand governor and recorded Ethernet offloads are retained for both modes.

Direct endpoints are 172.31.250.11 and .12. VPN endpoints are 10.77.1.10 and 10.77.2.10 on dummy interfaces, reached through `vpn0`; configured gateway .1 addresses remain reserved. Both modes use the same cable. Each Pi is an iperf endpoint as well as a VPN gateway, so physical CPU totals include endpoint work. This is a Pi-local test, not the separate laptop/AP forwarding demonstration. Direct mode precedes VPN mode in one ordered session.

Workloads and statistics

Every TCP point is a single stream, five transfers per path/direction, with two warm-up seconds omitted: 15 measured seconds native and 20 seconds physical. TCP goodput is receiver application bits per measured second. Figures and tables use the population standard deviation across five runs, not a confidence interval or across-session variability. UDP points are single observations; no repeat-based error bars are assigned. RTT is measured separately at idle and during TCP, with approximately the warm-up excluded from loaded summaries [3].

Whole-host/per-core `/proc/stat` counters are sampled every 0.5 s. Busy excludes idle/iowait without double-counting guest ticks. CPU equivalents are busy CPU-seconds per wall second, not architecture-normalized compute units. Softirq time and NET_RX/NET_TX counts are retained. Physical windows use synchronized UTC, with approximately one-sample alignment uncertainty. Pi temperature/frequency sampling is 0.5 s, throttling polling 5 s plus transfer completion. Idle remains separate, not subtracted.

Table 1. Measurement coverage.

Evidence lane	Transfers	Workload
Original unrestricted native	32	20 TCP, 2 UDP, 10 packet-size UDP points
Native CPU/capacity	160	100 TCP, 50 rate UDP, 10 packet-size UDP points
Physical Pi pair	32	20 TCP, 12 UDP; CPU on both Pis
Pi provisioning	Separate	Readiness, build and connectivity checks; no capacity estimate

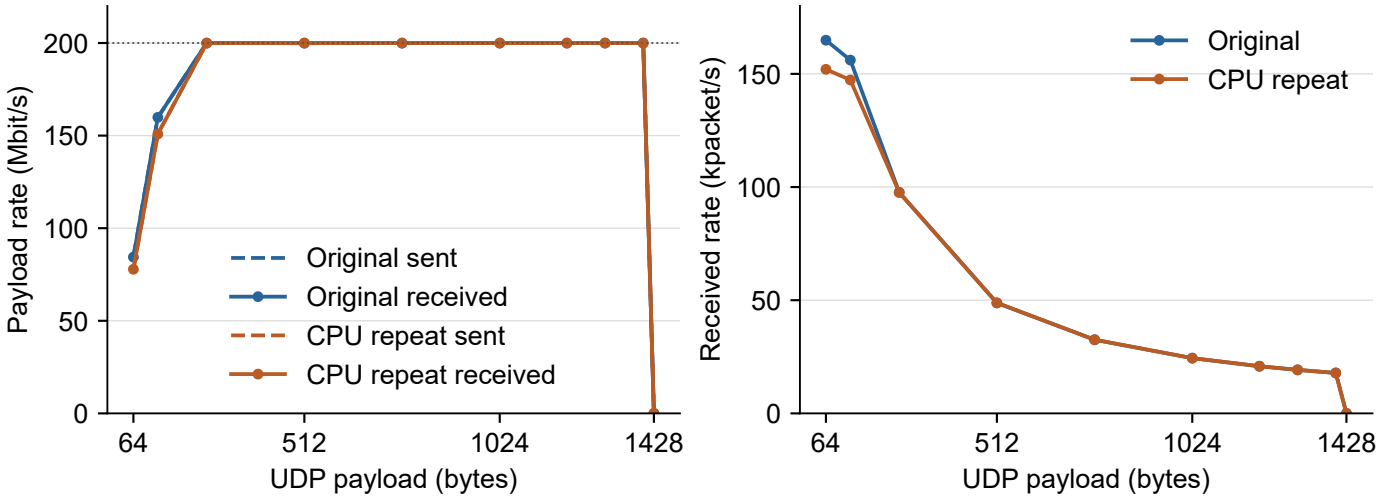


Figure 7. All ten native packet-size points in each session. Dashed lines are actual sent rates; solid lines are received rates. Requested payload rate is 200 Mbit/s. Zero delivery at 1428-byte UDP payload remains in the plot.

Unrestricted native software paths

Original TCP VPN gives $2295.1 \pm 44.6/2231.4 \pm 16.8$ Mbit/s forward/reverse; direct gives $62540.0 \pm 1288.0/62253.8 \pm 964.3$. The CPU repeat gives VPN $2302.74 \pm 63.19/2195.16 \pm 5.49$ versus direct $49802.34 \pm 1728.37/48423.91 \pm 374.92$. These virtual/offload paths do not represent physical link rates. Original idle RTT is 0.076/0.153 ms direct/VPN, versus 0.112/0.182 in the CPU repeat. Original 100 Mbit/s UDP delivers 99.99 Mbit/s with zero reported loss; the CPU VPN point also reports zero.

Packet size and sender limits

Original 64/128-byte points send 84.43/159.84 and receive 84.42/159.84 Mbit/s; CPU repeats send 77.90/150.91 and receive 77.83/150.85. Sender pacing/scheduling remain possible constraints. Payloads 256–1400 B receive near 200 Mbit/s. At 1428 B, delivery is zero and loss is unavailable, despite iperf's zero-valued field: inner headers exceed the unfragmented MTU.

Matched TCP capacities

At 100/500/1000 Mbit/s, forward direct gives 93.84, 470.72 and 941.49 versus VPN 89.42, 447.24 and 893.94 Mbit/s. Reverse is nearly identical: VPN reductions span 4.70–5.07%, consistent with useful payload shrinking under an outer-link budget. At 2500, direct gives $2353.89 \pm 0.05/2353.86 \pm 0.11$ and VPN $1823.05 \pm 24.09/1861.08 \pm 10.71$ Mbit/s. Retention falls to 77.45%/79.07%; processing, shaping and disabled offloads matter beyond fixed header cost.

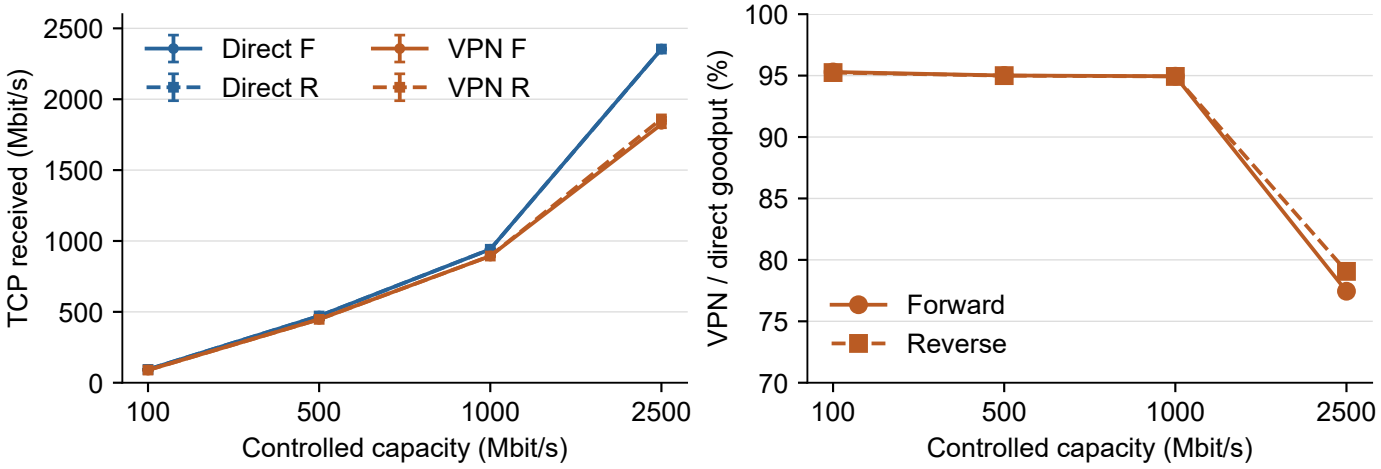


Figure 8. Native matched capacities: TCP receiver means with population SD over five transfers, and VPN/direct ratios of means. F/R indicate forward/reverse. Small error bars can be narrower than symbols.

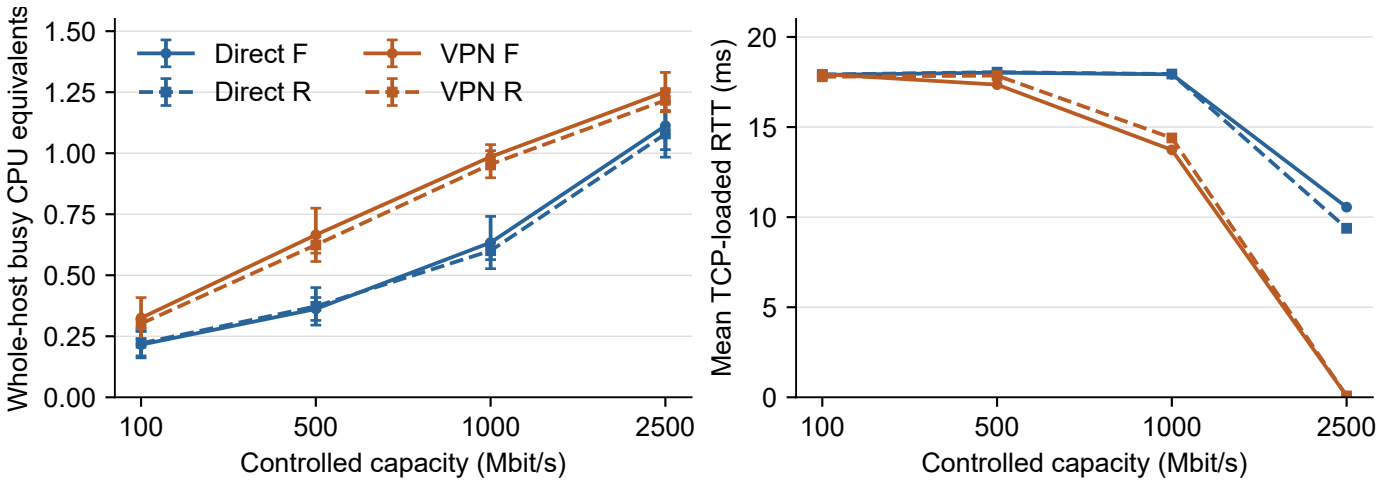


Figure 9. Native whole-host busy CPU equivalents and mean TCP-loaded RTT. CPU error bars show population SD over five transfers. Endpoint generation and background are included; idle is not subtracted.

CPU occupancy and queueing

Forward direct/VPN busy CPU equivalents are 0.22/0.32, 0.36/0.67 and 0.63/0.99 at 100/500/1000 Mbit/s; reverse follows the trend. At 2500, direct uses 1.11/1.08 and VPN 1.25/1.22 forward/reverse, with unequal delivered work. Unrestricted VPN uses 1.24 total equivalents, 1.00–1.02 softirq. VPN at 1000 uses 0.720/0.719 softirq versus direct 0.323/0.323. At 2500, VPN nears one softirq equivalent; busiest-core means are 62–68%, with 0.5 s samples reaching 100%. Migration and background variation prevent exact per-core bottleneck or idle-subtraction conclusions.

Controlled idle RTT is 0.105–0.195 ms. Loaded RTT is 17–18 ms at 100/500; at 1000, direct gives 17.94 and VPN 13.73/14.40 ms. At 2500, direct gives 10.56/9.38 and VPN 0.073/0.074 ms. The 20 ms TBF queues matter: low RTT at below-capacity goodput is not an equal-work advantage.

CPU usage across platforms

Whole-host percentages complement CPU equivalents and per-core occupancy. Figure 10 shows five-run TCP CPU means and population SD. Native VPN occupancy increases from about 1.4% at 100 Mbit/s to 4.4–4.5% at 1000 and 5.5–5.7% at 2500. The increase in CPU accompanies both higher delivered rates and per-packet work; the final capacity point also delivers less than the direct path.

On the Pis, transmitting VPN traffic uses about 25% host CPU and receiving uses 18–19%, whereas direct TCP remains below 5%. A Pi host has four logical CPUs and the native machine has 22, so these percentages have different denominators. They must not be read as equal computational cost across architectures. Whole-host measurements include endpoint generation, reception, instrumentation and background activity. A near-full transmitting core can coexist with a modest whole-host average; Fig. 12 makes that distinction visible.

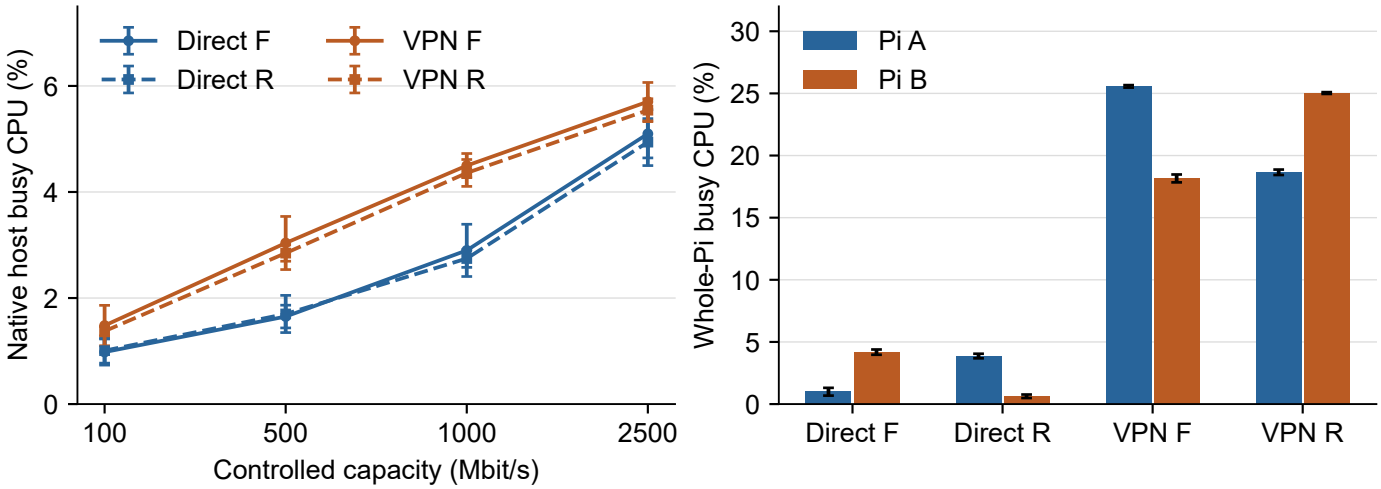


Figure 10. TCP CPU usage on the native host and both physical Pis. Means and population-SD error bars use five transfers per point. Native percentages span 22 logical CPUs; each Pi percentage spans four. F/R denote forward/reverse.

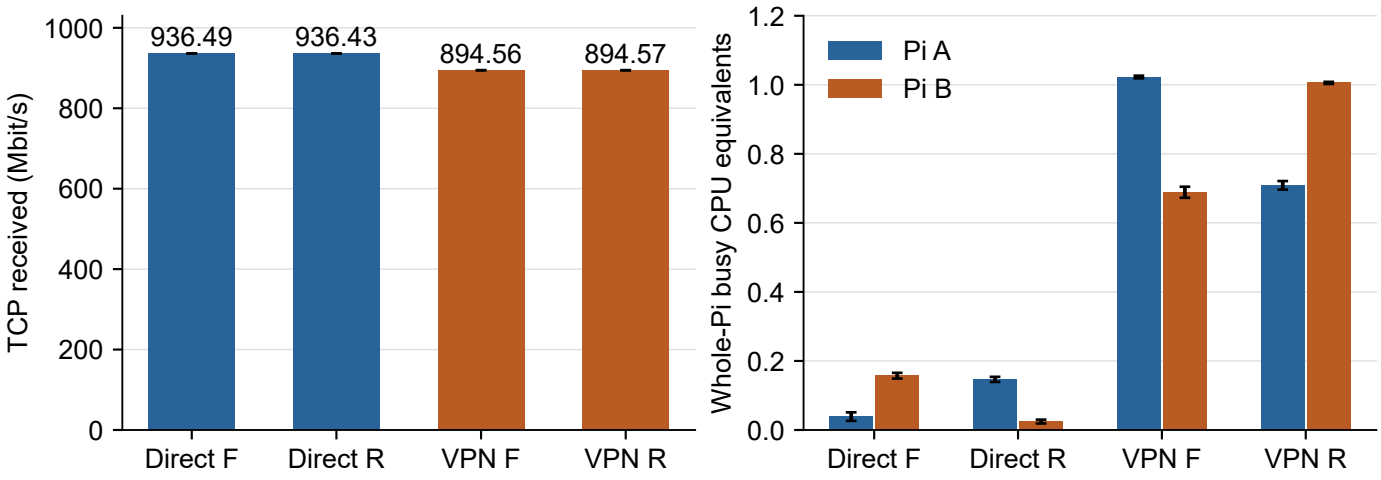


Figure 11. Physical 1 Gbit/s Ethernet TCP goodput and whole-Pi CPU occupancy. Each bar is the mean of five transfers; error bars are population SD. F/R indicate A-to-B/B-to-A traffic, while CPU colors identify the same physical Pi in both directions.

Physical TCP throughput and work distribution

Physical direct TCP gives 936.487 ± 0.002 Mbit/s forward and 936.432 ± 0.001 reverse. VPN gives 894.564 ± 0.029 and 894.568 ± 0.021 Mbit/s, retaining 95.52%/95.53% (4.48%/4.47% reduction). Small SD measures stability within one ordered session, not variation across days or configurations.

Forward VPN uses 25.57% host CPU on A (1.023 busy equivalents, 0.991 softirq) and 18.16% on B (0.689 equivalents, 0.444 softirq). Reverse shifts the heavier work to transmitting B: 25.03%, 1.006 equivalents, versus A's 18.65%, 0.709. The busiest transmitting core averages 98.91% forward and 96.26% reverse. About one-quarter host CPU thus coexists with a nearly saturated core.

Direct A/B CPU is 0.99/4.18% forward and 3.87/0.63% reverse. The VPN/direct difference includes encapsulation, crypto, packet work and different offload opportunities, with endpoint work in both. Idle is approximately 0.03% host CPU on both Pis.

Physical latency and delivery

Idle RTT from A is 0.162 ms direct and 0.197 ms VPN, an increase near 0.034 ms. Loaded forward means are 2.033 ± 0.059 ms direct and 4.215 ± 0.435 VPN; reverse gives 1.957 ± 0.132 and 4.080 ± 0.421 ms. Loaded probes originate on A in both traffic directions. This is a real cable path with recorded default queue/offload behavior.

No loaded probes are lost: direct is 1100/1100 forward and 1099/1099 reverse; VPN is 1095/1095 each direction. Counts include warm-up, unlike the RTT summaries. Direct TCP has zero retransmissions; VPN has 112 forward and 121 reverse across five runs each. These are recovered transport events, not application-data loss percentages.

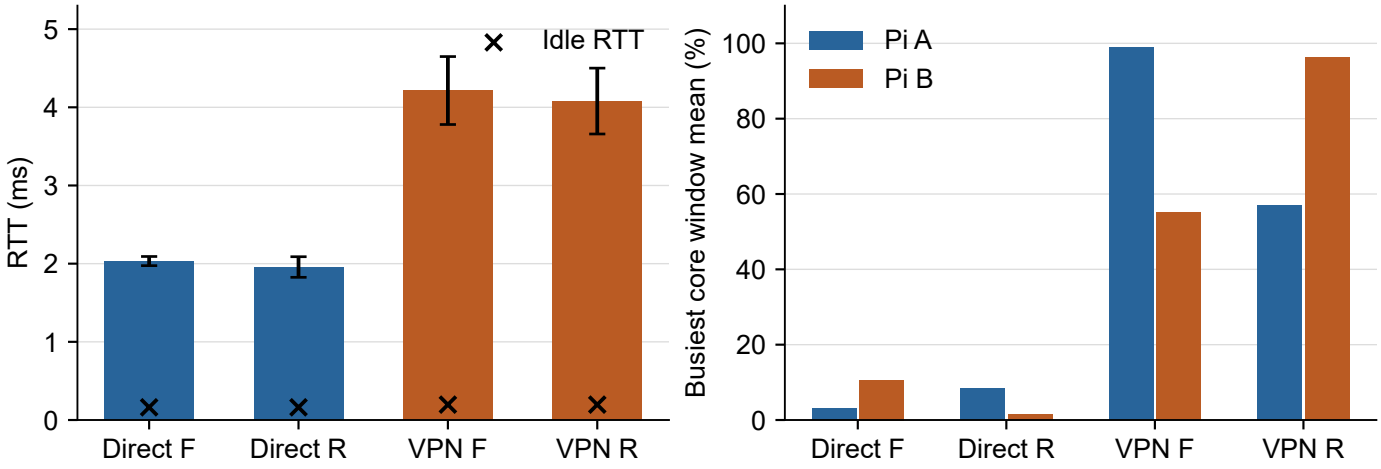


Figure 12. Physical idle/loaded RTT and busiest-core mean during TCP. RTT bars have population-SD error bars over five transfer means; crosses show the configuration's idle RTT. A busiest-core mean is a window statistic, not an instantaneous utilization reading.

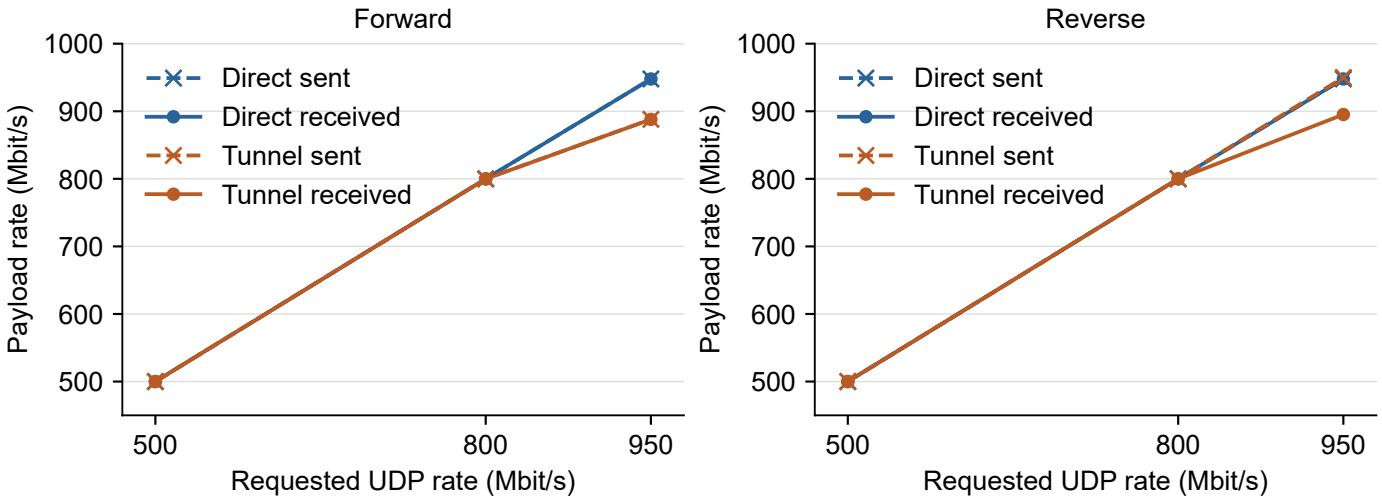


Figure 13. All twelve physical UDP observations. Sent and received rates nearly coincide except reverse VPN at 950 Mbit/s. Forward VPN misses that requested rate at the sender; reverse VPN reaches it and loses packets downstream.

Physical UDP and thermal evidence

Direct UDP delivers about 500, 800 and 947.8 Mbit/s in each direction with zero reported loss. VPN delivers almost 500/800 Mbit/s, with reported loss 0.0013–0.0021%. At requested 950 Mbit/s forward, it actually sends 888.03 and receives 888.00 Mbit/s (0.00130% loss); the target is missed. Reverse sends 950.00 and receives 895.19 Mbit/s with 4.73885% loss. A low forward loss rate cannot establish support for a 950 Mbit/s payload offer.

Temperature/frequency traces are included as a separate reproducible figure. Maximum recorded VPN TCP temperatures are 67.8 °C on A and 64.5 °C on B. All recorded throttling flags remain zero. The native and Pi validators pass 160 and 32 transfers respectively, including CPU evidence and matched controls. VPN-off probes fail before and after the physical experiment. Provisioning logs retain initial .1-address policy failures and loss on a temporary 100 Mbit/s USB-bridge underlay; these are setup diagnostics and are not pooled with final direct-cable results. The code contains unit, vector, namespace, lifecycle and concurrency suites; these archives do not constitute a newly executed full sanitizer/test qualification.

IX. Summary

VPTunnel implements an authenticated routed IPv4 path with a Rust Noise control plane and a kernel-owned encrypted record path. Explicit locators, per-generation counters, replay claims after authentication, current-policy admission and controller leases make security and lifetime boundaries visible in the design. The physical Pi experiment delivers approximately 895 Mbit/s TCP over gigabit Ethernet, retaining 95.5% of direct goodput; native matched tests show a similar percentage through 1 Gbit/s and a larger processing-related gap at 2.5 Gbit/s.

The evidence is limited to ordered sessions, four-peer design bounds, fixed endpoints, IPv4 and Pi-local physical endpoints. Per-core occupancy, loaded RTT and achieved UDP send rates constrain interpretation of headline goodput. Future work should complete and qualify conntrack-backed initiation witnesses in the fast path, profile per-packet hotspots, evaluate separate LAN hosts and concurrent peers, repeat randomized sessions with controlled frequency, and add independent security and failure-injection review. Host suspend/snapshot is outside the safe runtime envelope and requires fresh establishment.

X. Reproducibility

The GitHub repository contains source and evidence. Use disposable Linux 6.18.y, matching headers, GCC, Rust 1.93.1, Python, iproute2, nftables and iperf3. In `code/`, build with `./scripts/build.sh`; test with `sudo ./scripts/test.sh`; demonstrate with `sudo ./scripts/demo-netns.sh`. Analysis here validates saved evidence, not a new kernel-suite execution.

Unrestricted commands are `sudo ./scripts/bench-netns.sh` and `sudo ./scripts/bench-packetsize.sh`. Matched CPU/capacity runs use `sudo python3 tools/bench_capacity_cpu.py` with `--output results/new-native/run`. Apply `tools/summarize_capacity_cpu.py` and `tools/validate_capacity_cpu.py` to `results/new-native`.

Prepare native Pi builds, .10 aliases, a verified gigabit cable and SSH control sockets as in `docs/pi-benchmark.md`. Run `python3 tools/bench_pi_pair.py --output results/new-pi`, then `tools/summarize_pi_pair.py` and `tools/validate_pi_pair.py` on that directory. Keep both monitors and raw metadata. Report figures rebuild with `report/prepare_report.py`; `report/build.ps1` compiles XeLaTeX. Detailed setup and commands are in the GitHub reproduction guide.

References

[1] T. Perrin, “The Noise Protocol Framework,” revision 34, July 2018.

[2] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” RFC 8439, June 2018, doi: 10.17487/RFC8439.

[3] ESnet, “Invoking iperf3,” iperf3 documentation.

[4] Linux Kernel Developers, “Generic Netlink,” Linux 6.18 documentation.

Appendix A: Team Work Distribution and Member Rating

A.1 Work Distribution

The peer ratings below use the 1–10 scale and are unique, with no ties. The general work areas follow the team work plan.

#	Member Name	Roll No.	Tasks Done	Rating (1–10, unique)
1	Aayush Kuloor	24110004	Kernel tunnel runtime and integration; record encryption, replay protection, key lifecycle, resource accounting and teardown.	9
2	Anvit Aggarwal	24110048	Test infrastructure and namespace fixtures; CI, benchmark tooling, Raspberry Pi setup, demonstrations and experiment evidence.	9.5
3	Dhru Harsh Darshakbhai	24110107	Forwarding policy and ACLs; fragment normalization, IP checks, MTU/ICMP handling and policy/concurrency tests.	8.5
4	Raj Amit Modi	24110282	Rust control plane; configuration and identities, Noise key exchange, Netlink client, controller lifecycle and control-plane tests.	10

Since no minimum difference value was mentioned, we are taking it as 0.5—the difference value mentioned during Project 1 evaluation.

Appendix B: AI Usage Details

B.1 Prompts

AI assistance covered learning, planning, implementation, testing and artifact preparation. The team reviewed the suggestions, made implementation decisions and retained responsibility for the submitted work. The division of work was as follows.

Stage	AI tool and contribution	Team contribution
Learning and ideation	ChatGPT Web helped explain the networking and cryptographic concepts and suggested ideas for the security design.	We discussed the suggestions and decided the project scope.
Implementation planning	Codex helped develop the architecture and implementation plan.	We reviewed and approved the plan and component boundaries.
Coding	Codex and Claude Code were used for all project coding, including the VPN implementation, control plane, packet policy, IP handling, supporting scripts and related files.	We reviewed the implementation and made adjustments where required.
Testing	Codex helped create and run tests, investigate failures and revise the implementation.	We checked that the resulting implementation worked in the intended environments.
Benchmarking	AI was not responsible for carrying out the benchmark measurements.	We ran the benchmarks and collected the experimental results.
Report	Codex created the complete report from the implementation and collected results, including plotting and layout.	We reviewed the report and made the final changes and edits.
Presentation	ChatGPT Work created the complete presentation using GPT-5.6 Sol.	We reviewed the presentation and made the final changes and edits.

Prompts, response records and further details are available in the repository's AI usage folder on GitHub.